03

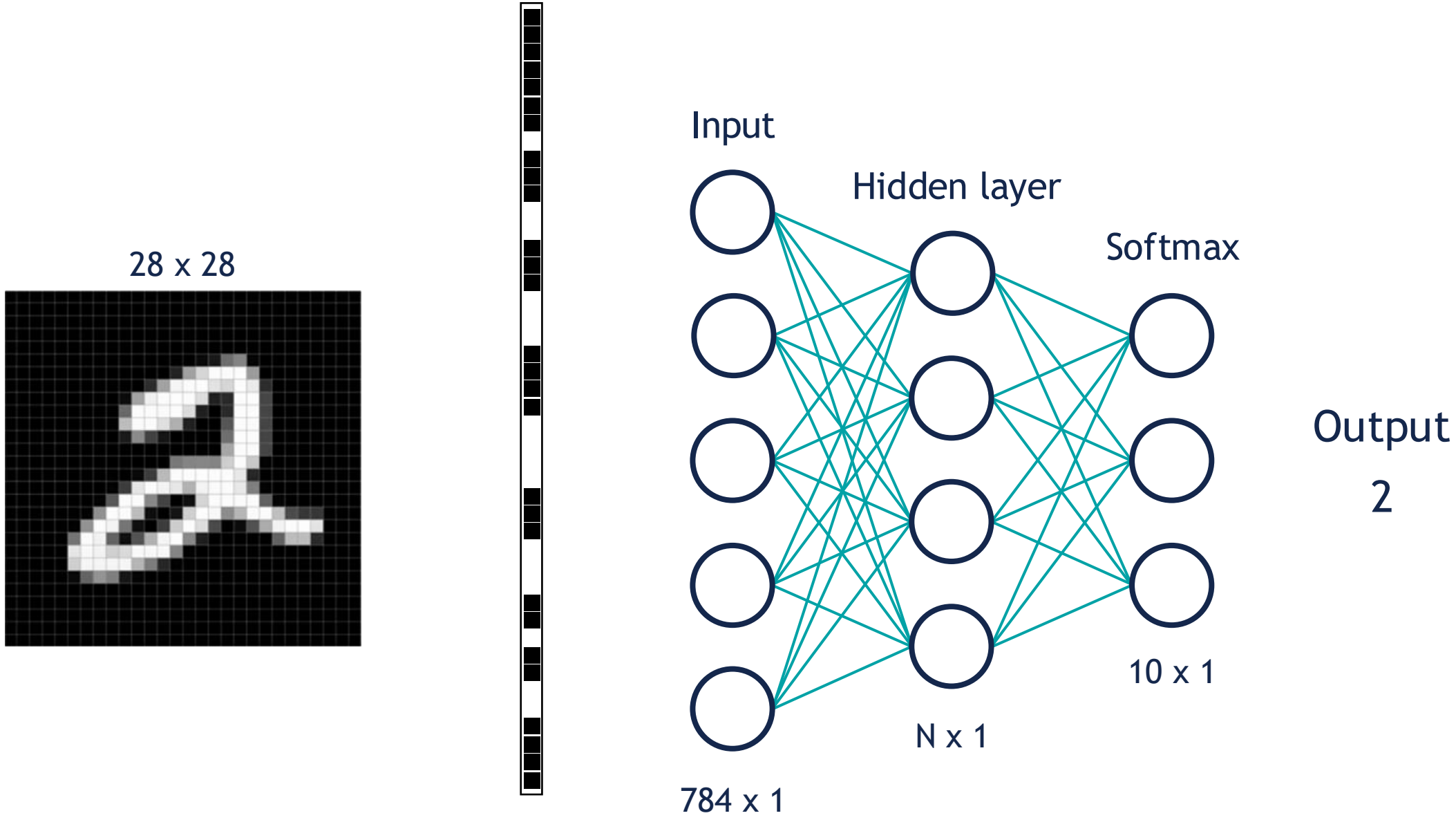
Convolutional Neural Networks

Introduction to CNNs

Kernel convolution

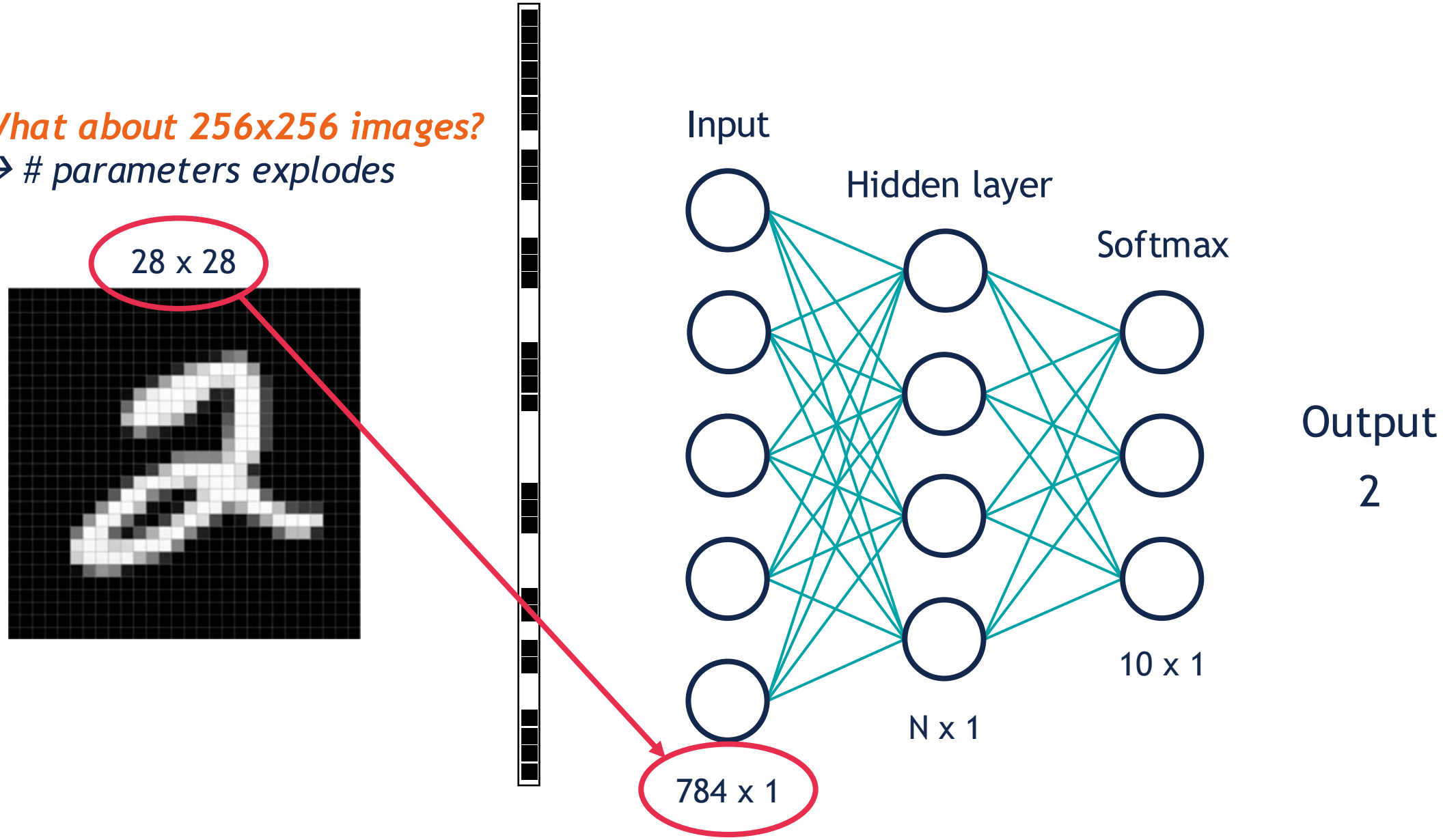
Channels and dimensions

CNN architectures


28 x 28
Input
Hidden layer
Softmax
Output
2
10 x 1
N x 1
784 x 1

What about 256x256 images?

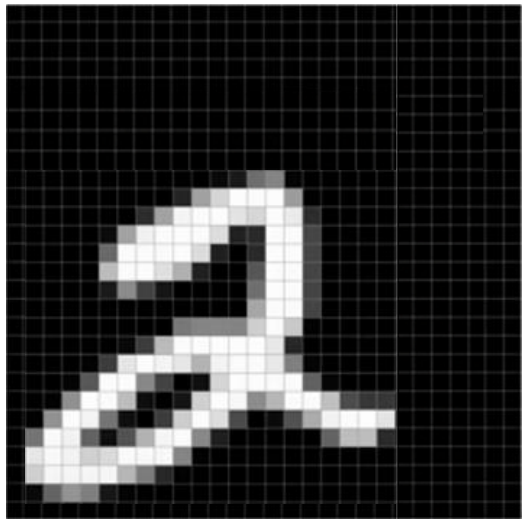
→ # parameters explodes

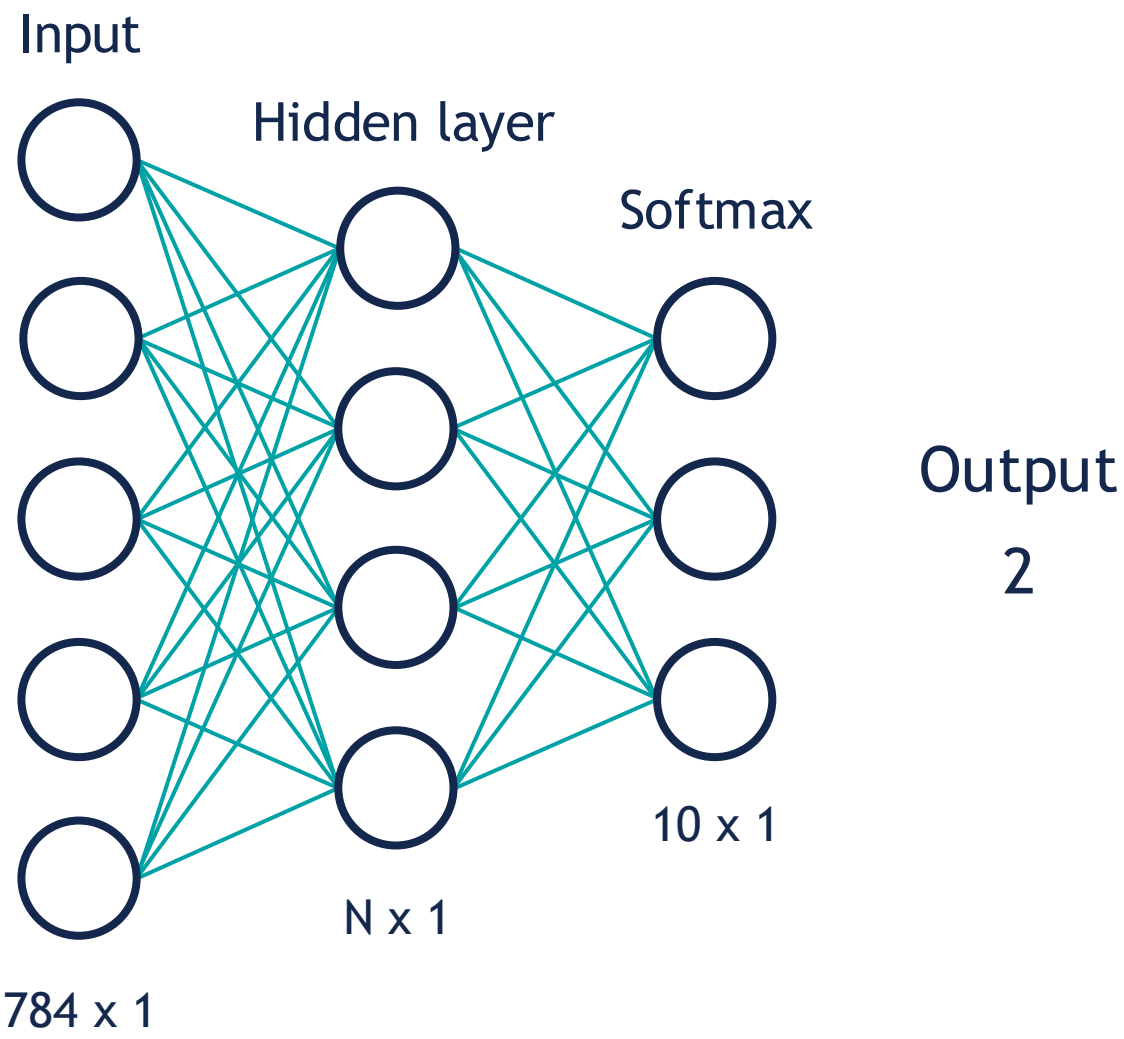


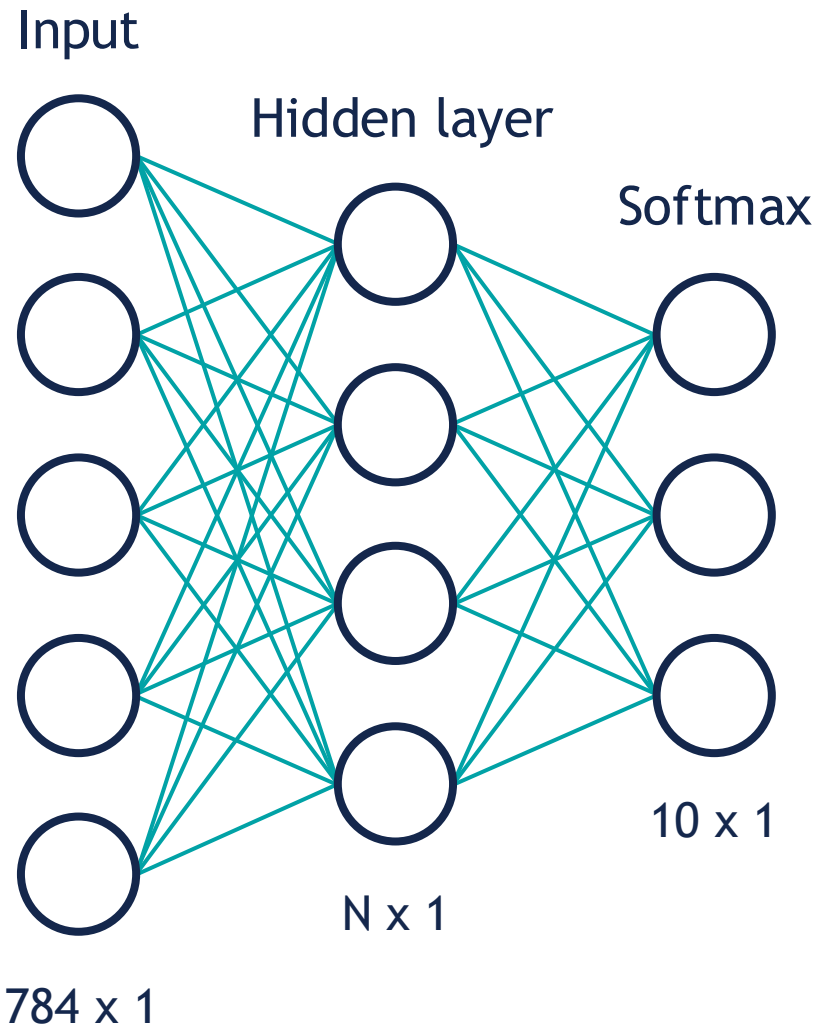
What if the number has a different position?

→ needs to relearn the same pattern with different neurons

28 x 28







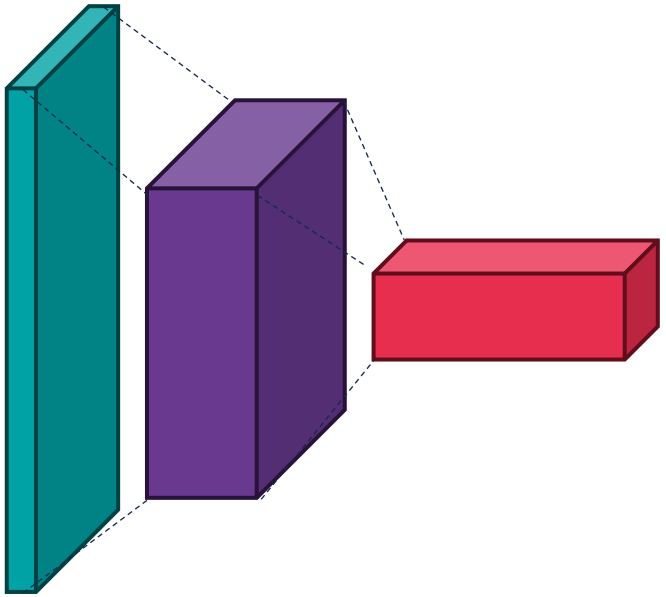
Output

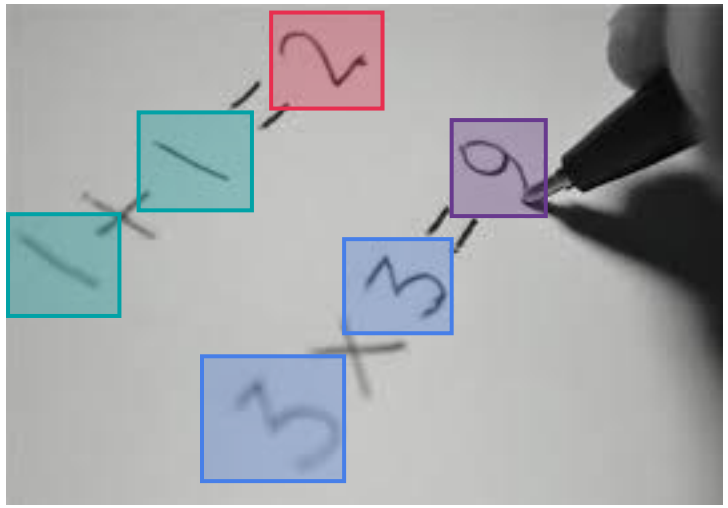
???

https://medium.com/@koushikkushal95/mnist-hand-written-digit-classification-using-neural-network-from-scratch-54da85712a06

Convolutional Neural Network

(specialized architecture for computer vision)





Output

1, 1, 2, 3, 3, 9

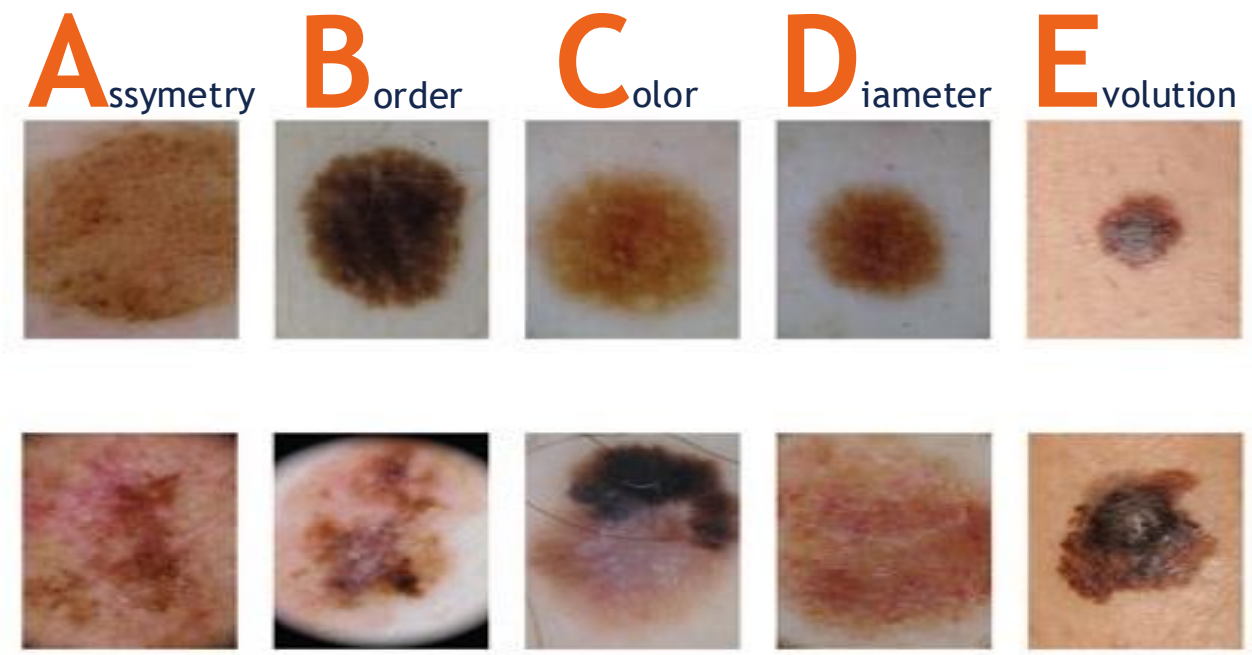
https://medium.com/@koushikkushal95/mnist-hand-written-digit-classification-using-neural-network-from-scratch-54da85712a06

What is computer vision?

Computer vision

Computer vision = digital systems for detection and processing of visual information

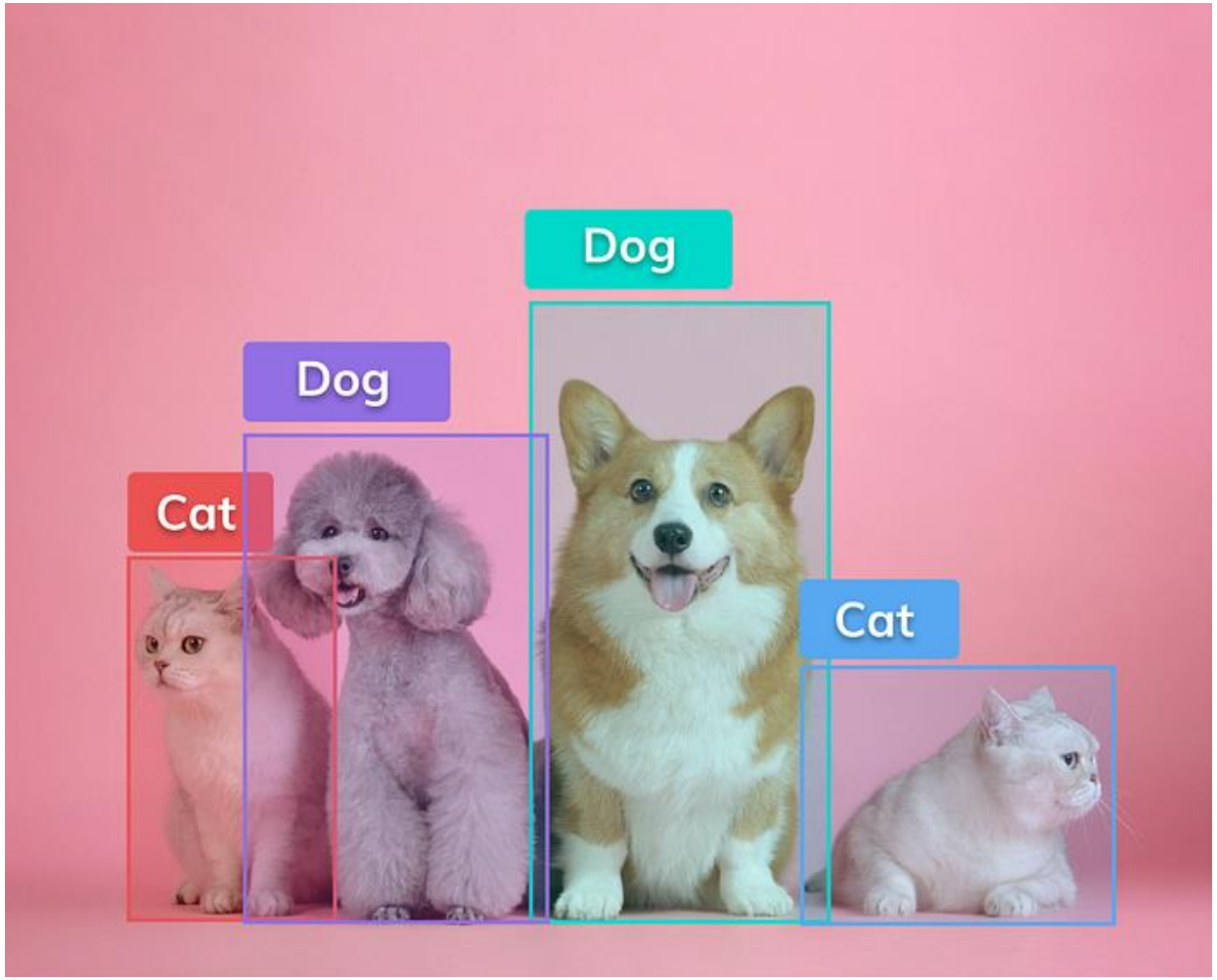
- Classification
  what is in the image?



Computer vision

Computer vision = digital systems for detection and processing of visual information

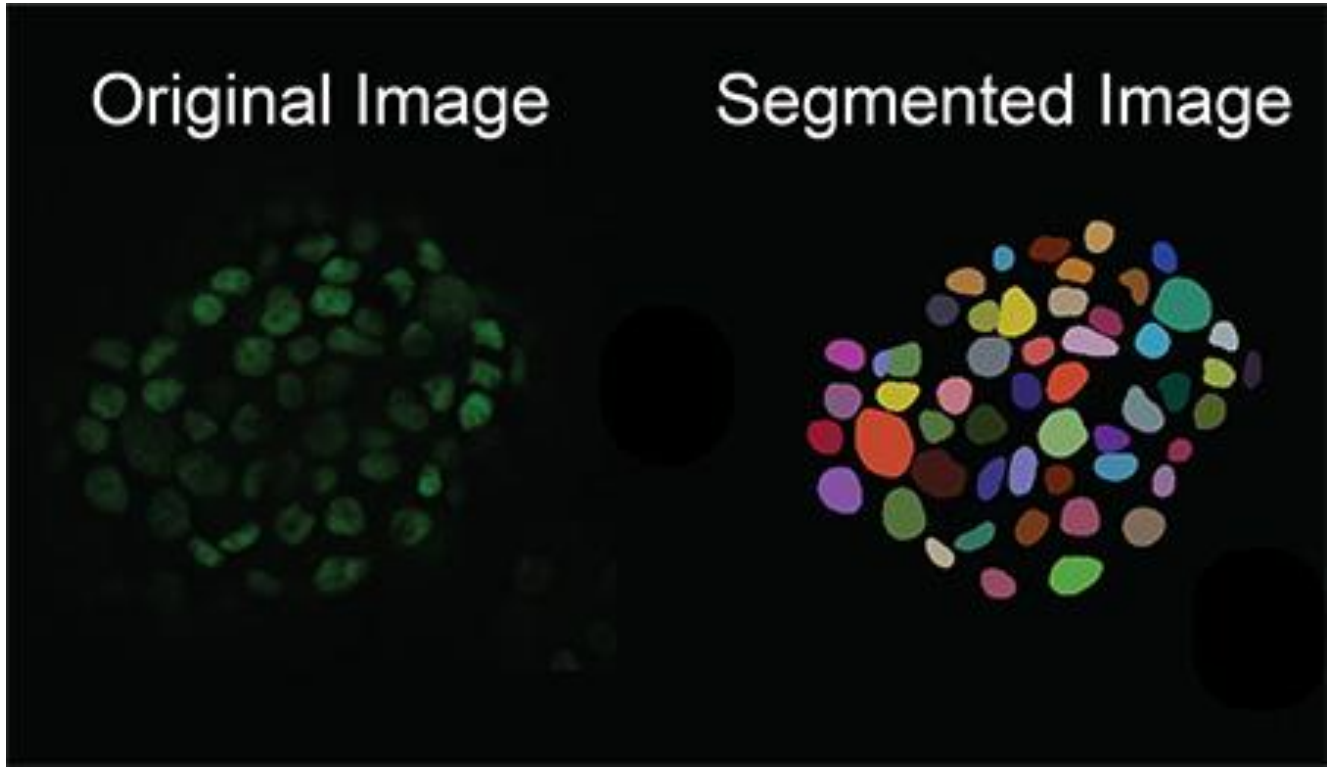
- Classification
  what is in the image?
- Object detection/segmentation
  where is it?



Computer vision

Computer vision = digital systems for detection and processing of visual information

- Classification
  what is in the image?
- Object detection/segmentation
  where is it?

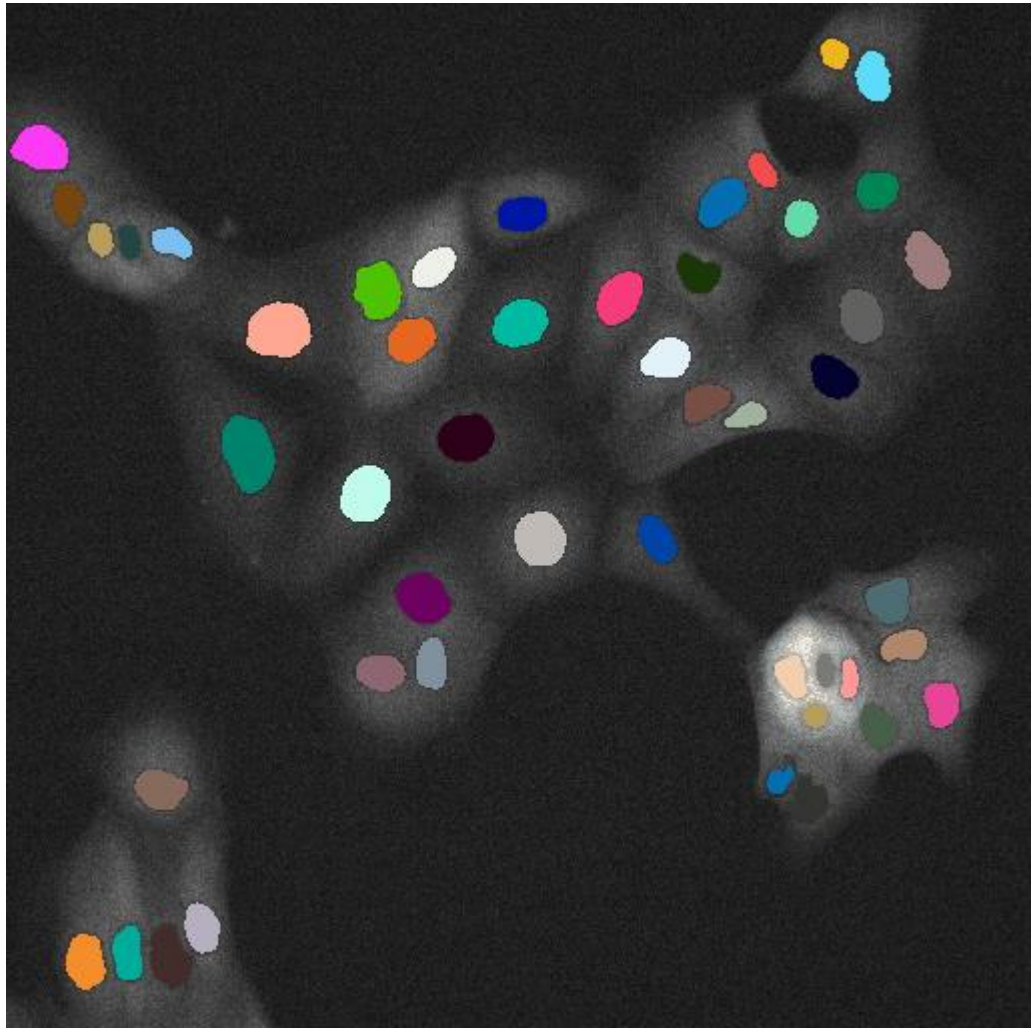


UC Santa Cruz

Computer vision

Computer vision = digital systems for detection and processing of visual information

- Classification
  what is in the image?
- Object detection/segmentation
  where is it?
- Object tracking
  where is it going?



Computer vision

Computer vision = digital systems for detection and processing of visual information

- Classification
  what is in the image?
- Object detection/segmentation
  where is it?
- Object tracking
  where is it going?

Introduction to CNNs

Kernel convolution

Channels and dimensions

CNN architectures

Why do we need kernels?

206	205	247	245	244	253	247	245	136	151	255	255	255	255	255	255	234	207	231	255	254	254	255	255	254	255	252	255	255	254	255	247
244	161	137	244	254	255	254	255	118	103	209	228	155	153	236	193	74	52	66	173	255	254	254	255	255	255	254	255	254	253	244	184
192	154	75	200	249	255	255	255	110	96	84	61	35	44	89	53	44	45	43	54	140	213	253	255	255	255	255	245	187	186	176	223
90	109	96	143	223	255	255	252	117	75	41	35	31	24	25	36	45	44	44	46	81	118	148	234	252	254	255	248	231	248	255	254
67	69	107	196	236	255	255	255	104	25	34	35	29	20	25	34	32	30	32	34	53	85	100	142	231	242	247	249	255	255	255	255
55	51	45	134	218	251	255	232	51	12	26	33	24	24	46	75	82	78	71	66	58	53	67	90	136	228	208	158	253	246	249	255
79	58	56	75	224	255	255	118	11	27	74	99	91	106	140	162	173	173	173	172	158	137	92	46	78	187	217	206	254	222	233	255
38	43	47	52	147	255	229	56	41	81	129	145	160	169	169	172	178	179	178	179	177	177	172	110	31	82	209	238	255	244	249	255
40	40	33	36	90	245	171	32	65	110	139	145	151	162	171	174	178	179	182	184	187	183	173	162	71	45	167	255	254	255	254	255
37	44	44	31	69	250	158	36	70	129	143	142	153	162	171	175	177	178	182	191	194	188	180	170	120	51	137	255	254	250	254	255
34	45	51	64	116	237	181	53	116	138	140	143	154	164	176	178	174	177	183	186	185	185	183	178	140	66	141	254	252	225	249	255
34	36	52	74	71	188	156	63	131	134	144	155	160	161	173	179	178	179	189	193	190	185	187	182	156	93	148	250	254	214	247	255
32	38	52	54	159	250	126	57	129	138	138	140	151	156	166	168	171	178	180	187	186	185	185	183	180	102	136	242	255	255	254	254
36	32	72	129	212	228	115	65	121	104	102	104	94	103	134	158	170	162	125	108	121	143	155	190	191	104	134	230	253	253	255	251
61	82	116	107	179	247	124	60	101	90	111	119	103	81	94	147	191	178	126	98	123	153	147	161	200	92	100	222	207	167	227	215
144	178	167	231	210	232	170	67	115	88	76	62	83	85	88	139	192	190	135	80	53	99	141	165	201	97	79	192	245	235	248	249
127	145	149	195	204	213	197	95	133	122	117	133	126	108	110	139	191	197	167	129	127	148	147	171	188	110	121	228	233	180	215	212
87	112	100	79	85	82	65	75	142	148	151	153	138	125	120	149	191	190	193	175	174	193	198	190	208	127	163	239	219	149	198	195
63	83	109	134	129	106	39	78	132	142	155	159	139	111	124	164	195	200	186	192	191	195	200	202	200	143	217	253	249	242	238	234
69	78	78	113	97	74	43	106	127	140	152	155	125	97	112	150	185	194	174	183	196	198	202	208	209	166	247	254	255	254	254	254
72	44	63	59	46	52	49	74	127	137	146	149	132	103	78	90	134	141	168	165	199	207	204	203	216	193	236	244	251	242	236	243
55	20	69	73	59	80	46	74	117	127	144	161	148	124	105	120	156	187	193	162	189	206	201	205	214	194	174	185	197	188	183	193
65	49	77	89	50	68	43	61	109	127	141	147	113	100	121	145	148	169	181	176	181	201	201	205	202	174	166	169	178	183	188	184
82	76	92	79	54	58	37	47	90	121	132	116	89	78	111	146	163	149	122	124	180	197	197	198	178	149	146	152	155	157	159	168
104	107	122	123	105	79	27	33	66	111	122	120	114	114	147	175	190	196	183	101	170	200	187	185	156	146	145	139	137	141	140	145
117	124	127	133	135	105	21	28	37	88	115	121	128	128	141	142	168	202	212	153	164	186	180	168	154	146	144	149	151	151	147	144
119	118	118	125	128	111	21	29	28	58	100	118	131	140	151	159	186	201	205	192	180	168	149	166	119	144	147	143	140	141	144	148
117	119	125	130	139	106	18	29	44	58	70	102	133	147	168	197	212	215	210	195	177	152	133	195	57	59	126	151	145	143	142	141
115	123	126	134	145	102	27	54	52	38	45	69	105	135	175	189	193	216	206	166	139	111	164	203	74	5	121	151	142	142	143	146
101	108	123	121	132	105	44	40	31	35	57	44	58	101	147	144	138	163	145	94	90	145	196	187	84	48	165	160	142	144	142	145
98	97	97	96	104	76	34	33	30	48	41	49	51	58	74	53	55	66	63	89	150	188	209	156	62	108	140	149	125	133	131	131
102	102	97	88	73	35	30	23	42	50	65	41	90	60	59	51	57	82	123	157	187	205	189	62	96	151	105	101	154	135	130	129

Why do we need kernels?

Image as a flattened array

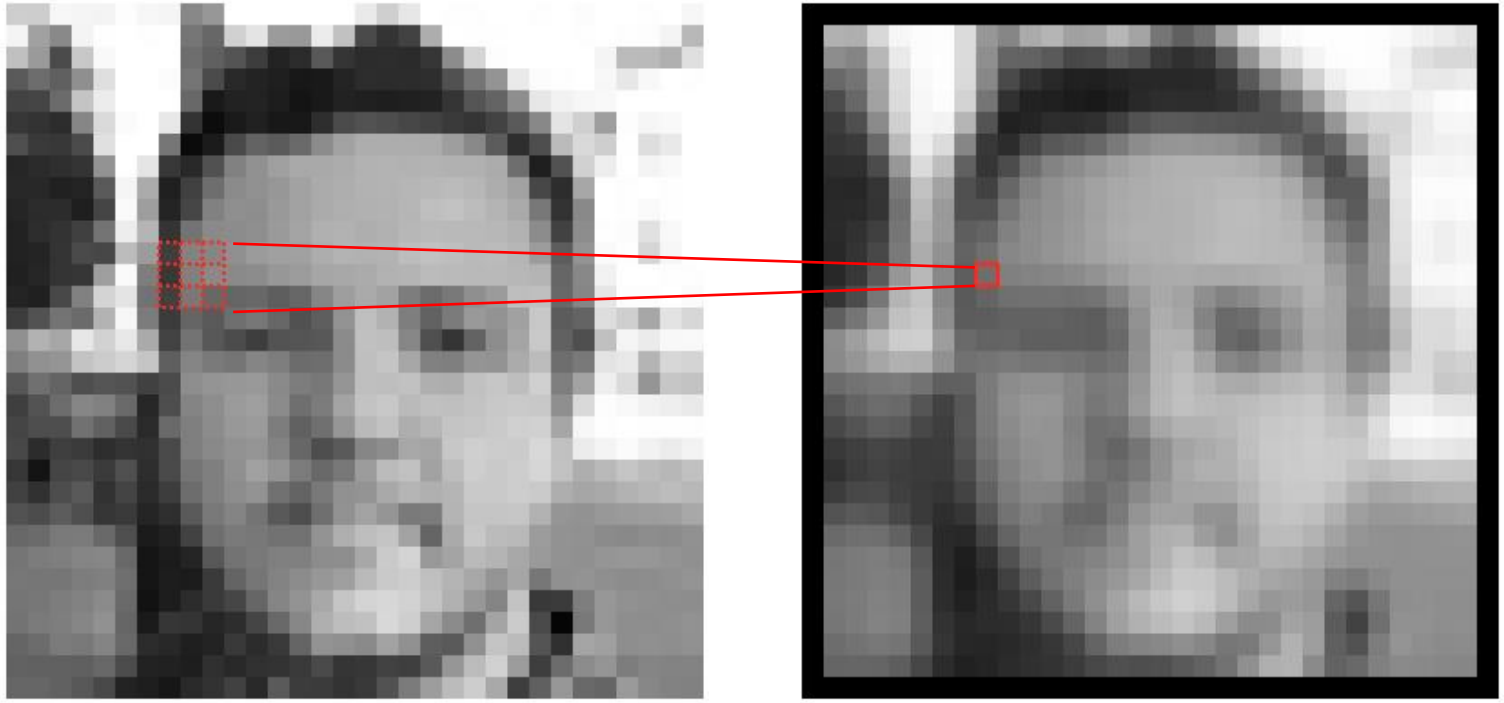
- Parameter explosion
  (e.g. 224x224x3 → ~150k features)
- Loss of spatial structure
  nearby pixels matter more than distant ones
- Can't handle invariance

Why do we need kernels?

Image as a flattened array

- Parameter explosion
  (e.g. 224x224x3 → ~150k features)
- Loss of spatial structure
  nearby pixels matter more than distant ones
- Can't handle invariance
  = objects should be recognized independent of how they appear (position, size, rotation, …)

Kernel convolution



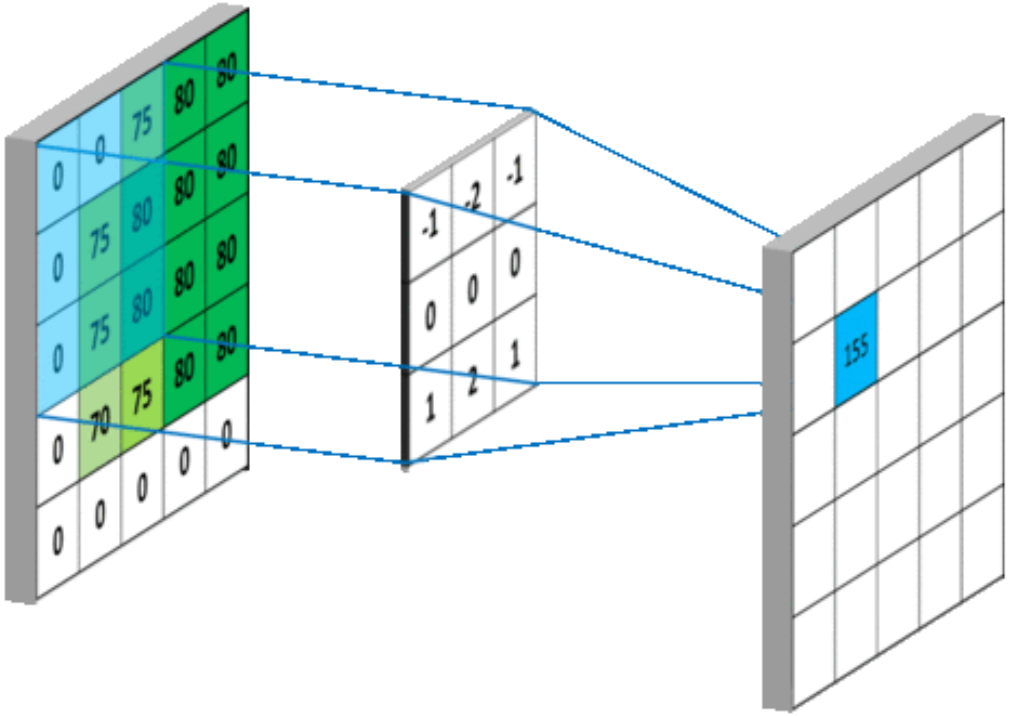
Kernel (aka filter) = small matrix that is moved across the image

- Image processing (blurring , edge detection, etc.)
- Feature extraction

Kernel convolution

Original	*Gaussian Blur*	*Sharpen*	*Edge Detection*
$\begin{bmatrix} 0 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 0 \end{bmatrix}$	$\frac{1}{16}\begin{bmatrix} 1 & 2 & 1 \\ 2 & 4 & 2 \\ 1 & 2 & 1 \end{bmatrix}$	$\begin{bmatrix} 0 & -1 & 0 \\ -1 & 5 & -1 \\ 0 & -1 & 0 \end{bmatrix}$	$\begin{bmatrix} -1 & -1 & -1 \\ -1 & 8 & -1 \\ -1 & -1 & -1 \end{bmatrix}$

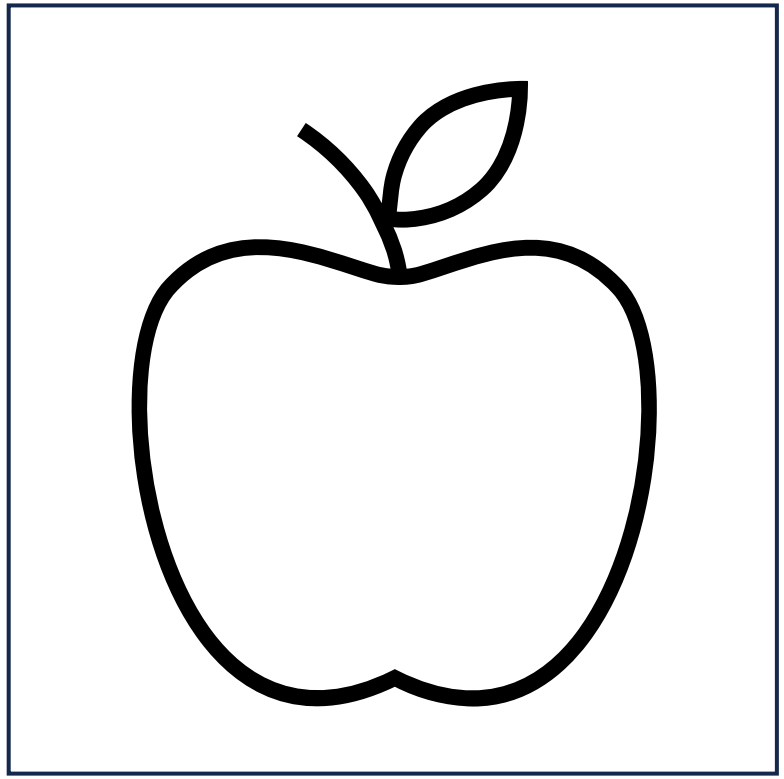
Kernel convolution



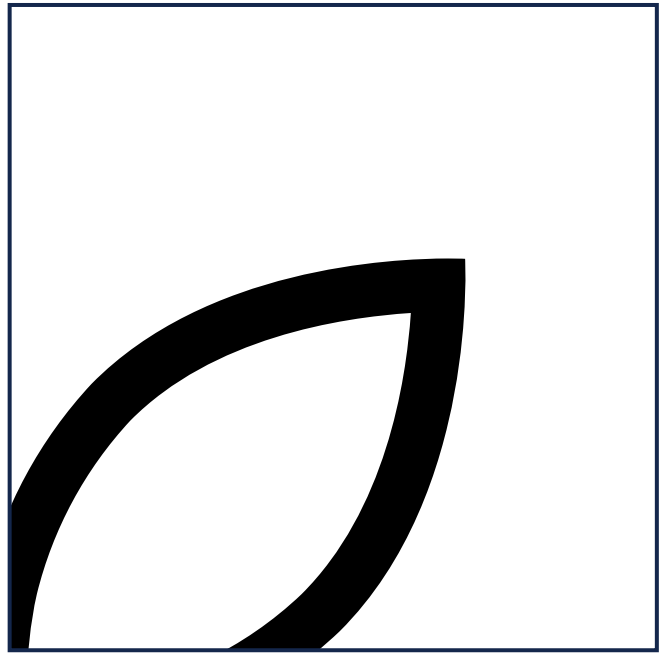
- Learn the filter values (small matrix)
  → reduced parameter count
- Combines information from nearby pixels into one value
  → preserves spatial structure
- Is moved across the image
  → handles invariance
- Detects patterns like edges, shapes, texture, etc.

VIB

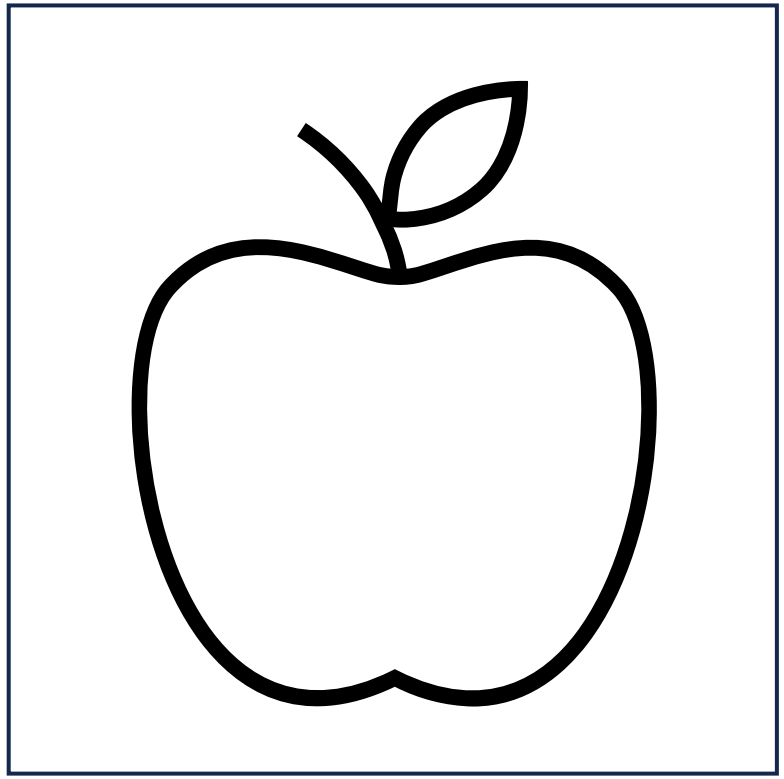
Kernel convolution



“Leaf kernel”



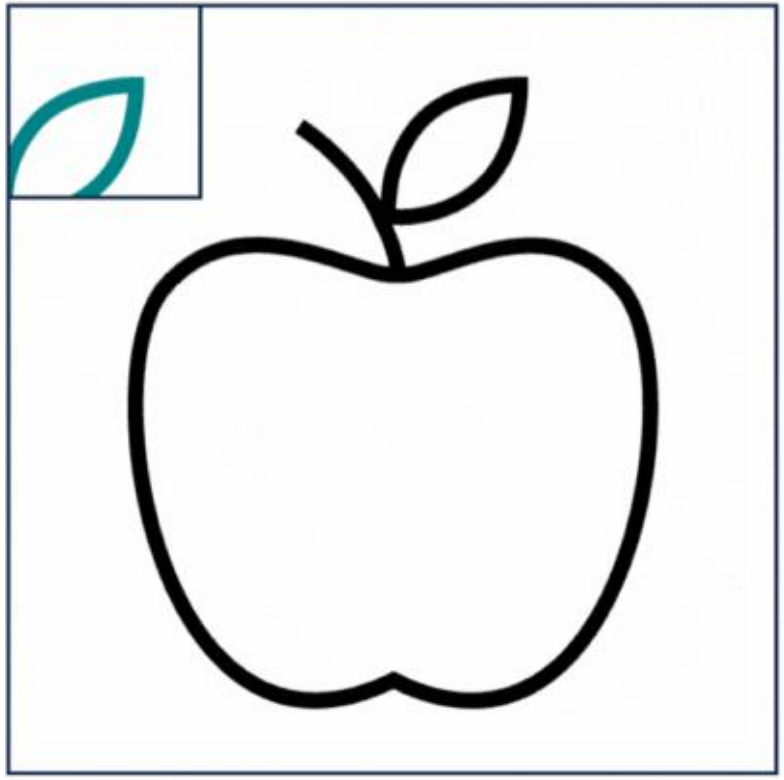
Kernel convolution



“Leaf kernel”

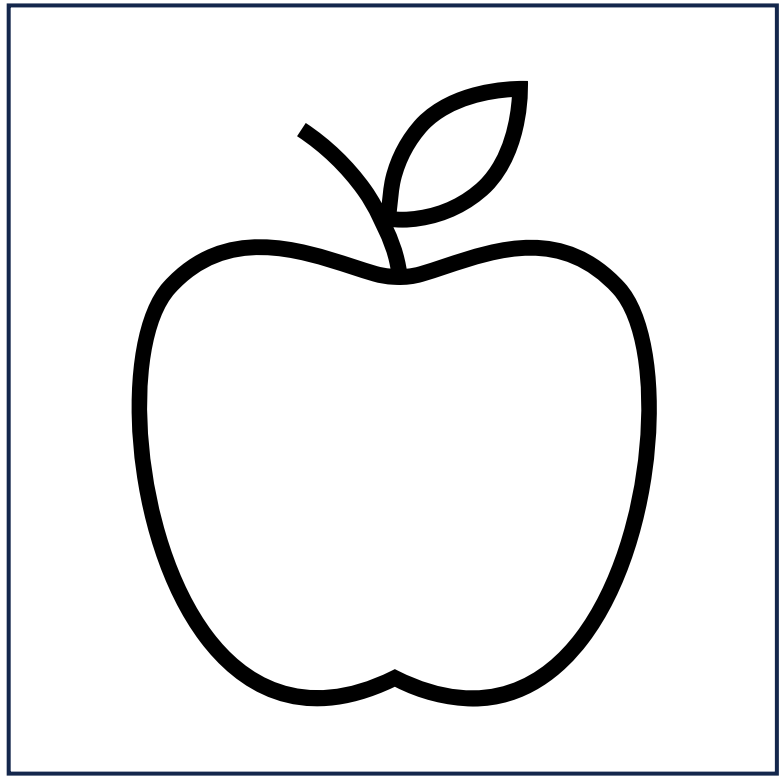
0	0	0	0
0.1	0.4	0.5	0
0.5	0.1	0.5	0
0.2	0.2	0.4	0

Kernel convolution



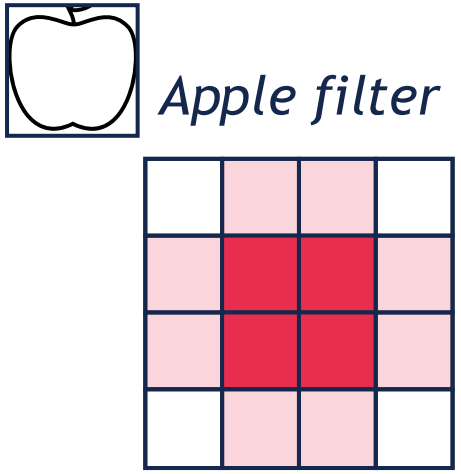
0			

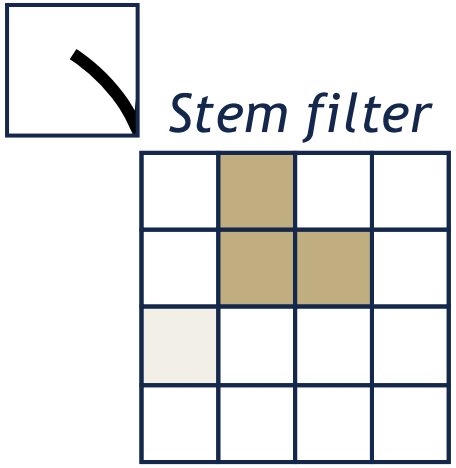
Kernel convolution



"Leaf detected"

0	0.1	1	0
0	0	0	0.2
0.1	0	0	0.1
0	0.2	0.2	0





We can increase the accuracy by **adding more filters** (~ features)

Kernel convolution

7	2	3	3	8
4	5	3	8	4
3	3	2	8	4
2	8	7	2	7
5	4	4	5	4

*

1	0	-1
1	0	-1
1	0	-1

7x1+4x1+3x1+
2x0+5x0+3x0+
3x-1+3x-1+2x-1
= 6

=

6		

https://medium.com/nerd-for-tech/all-about-convolutions-331471b9e5e5

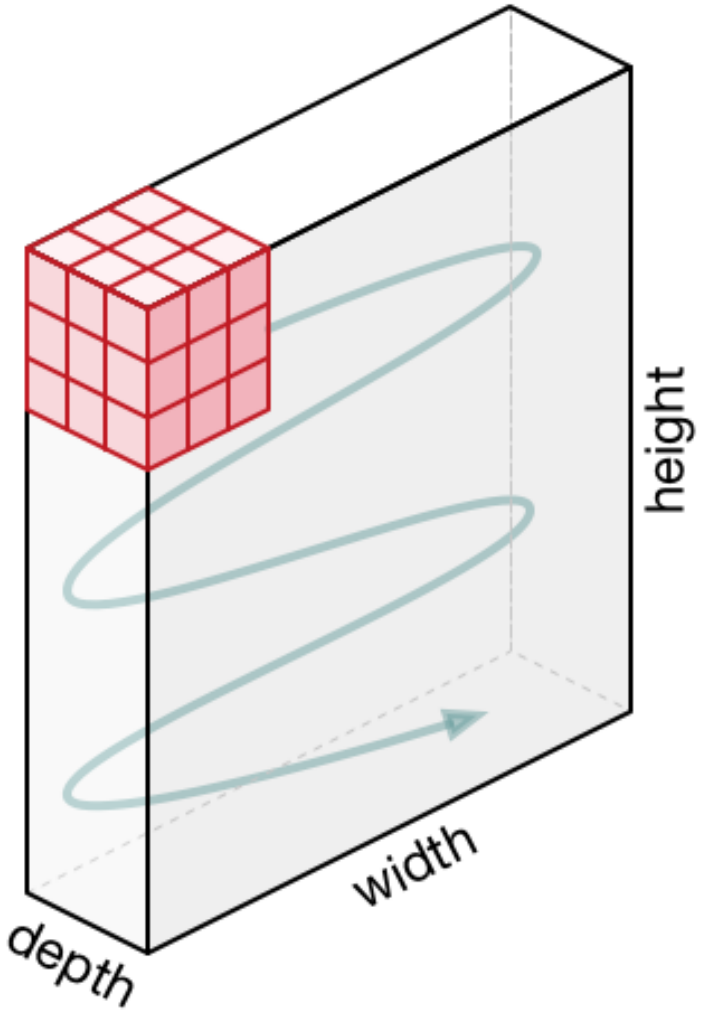
Introduction to CNNs

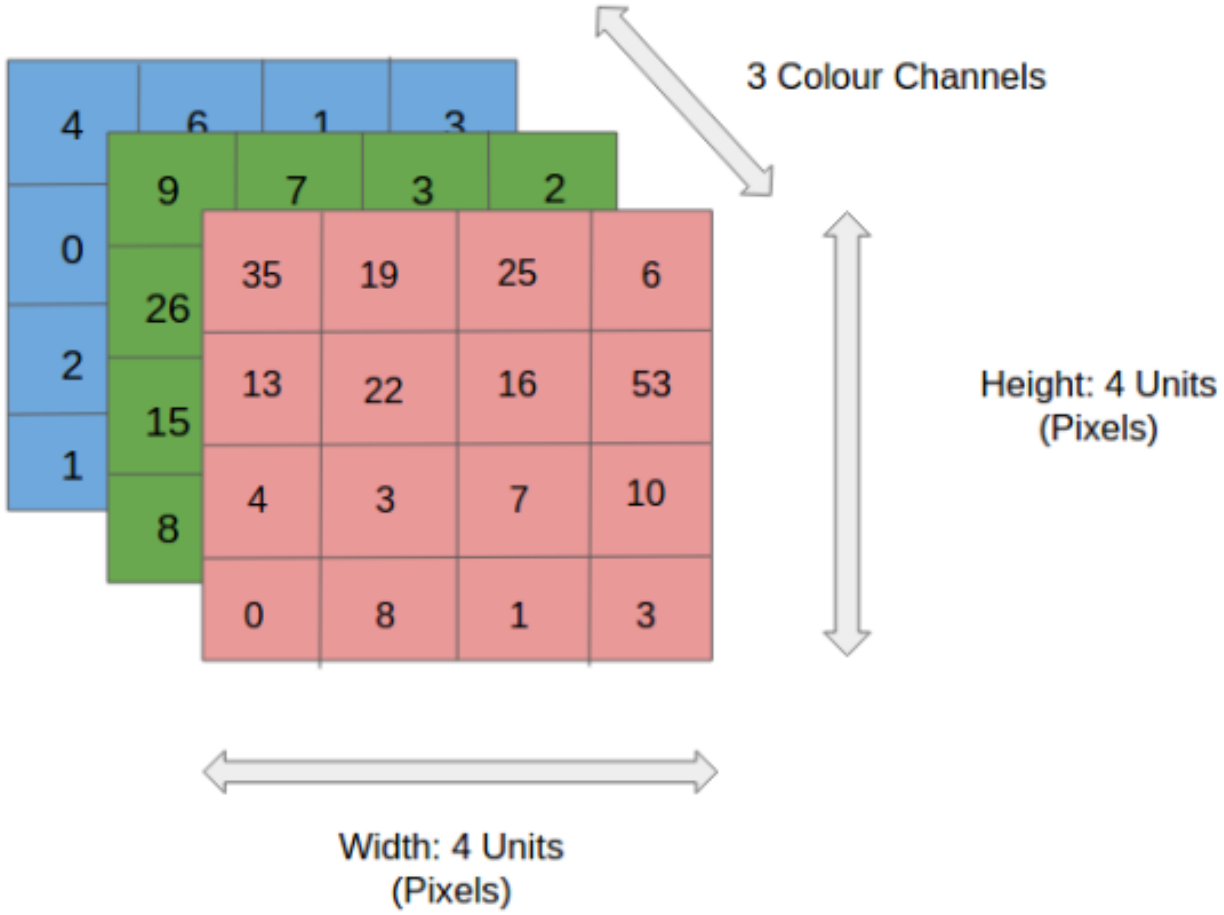
Kernel convolution

Channels and dimensions

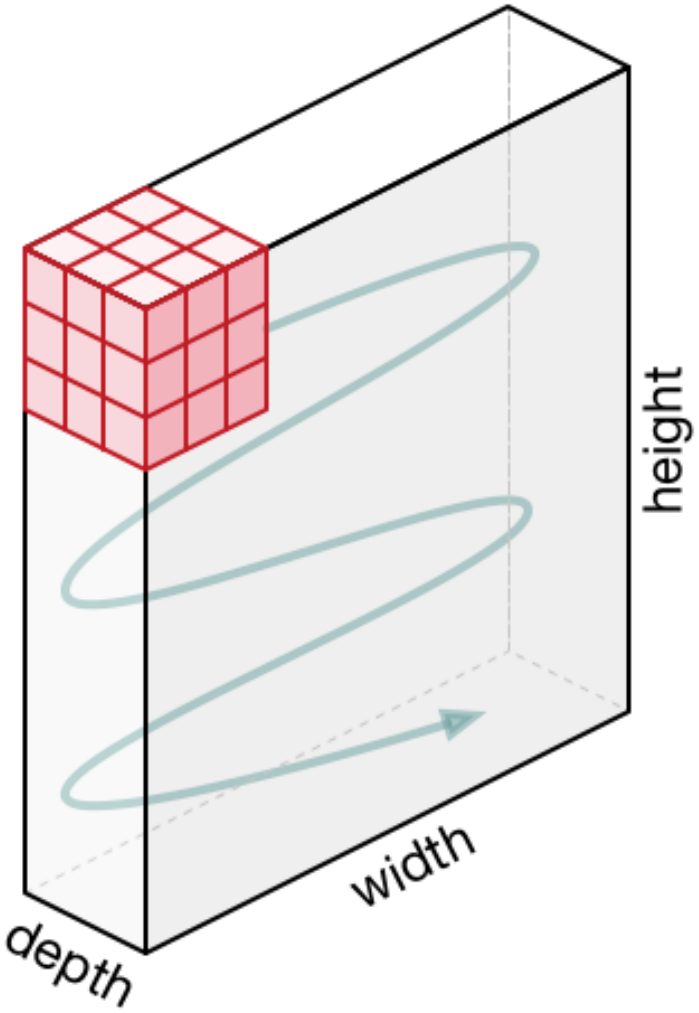
CNN architectures

Channels & dimensions





Channels & dimensions



Square kernels

→ width = height

Depth = number of channels from the previous layer

- Input: RGB image (3 channels)
  → first layer: kernel size (3xNxN)
- First hidden layer: 10 kernels
  → second layer: kernel size (10xNxN)

Kernel convolution

7	2	3	3	8
4	5	3	8	4
3	3	2	8	4
2	8	7	2	7
5	4	4	5	4

*

1	0	-1
1	0	-1
1	0	-1

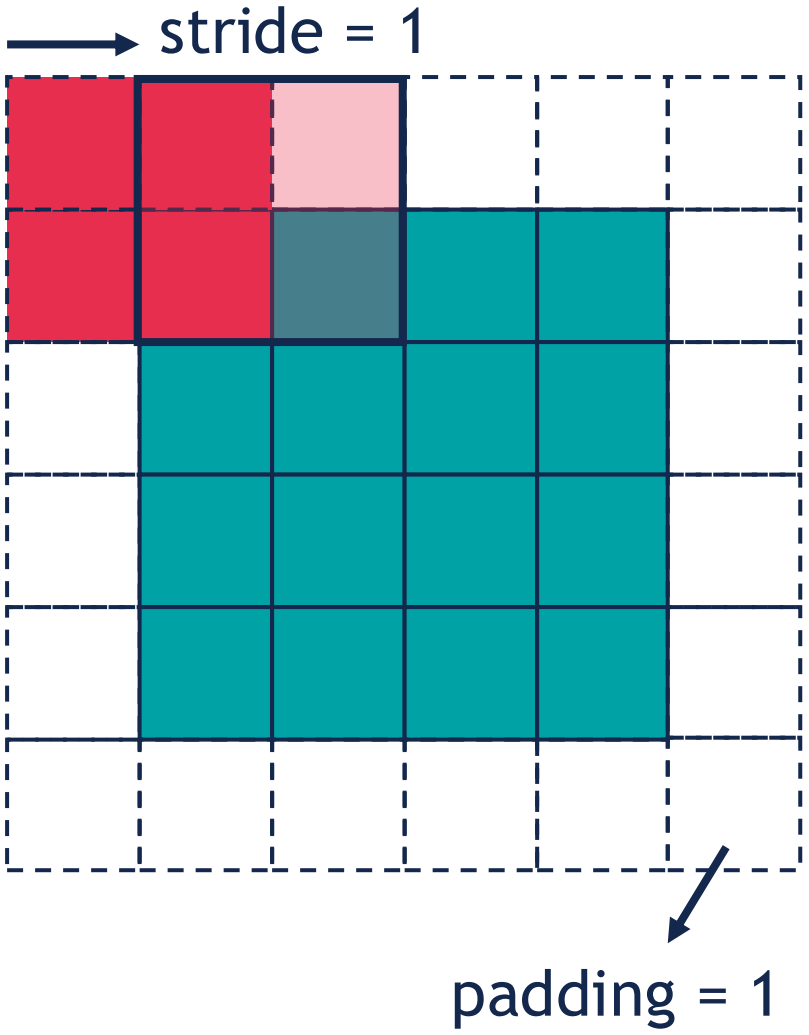
7x1+4x1+3x1+
2x0+5x0+3x0+
3x-1+3x-1+2x-1
= 6

=

6		

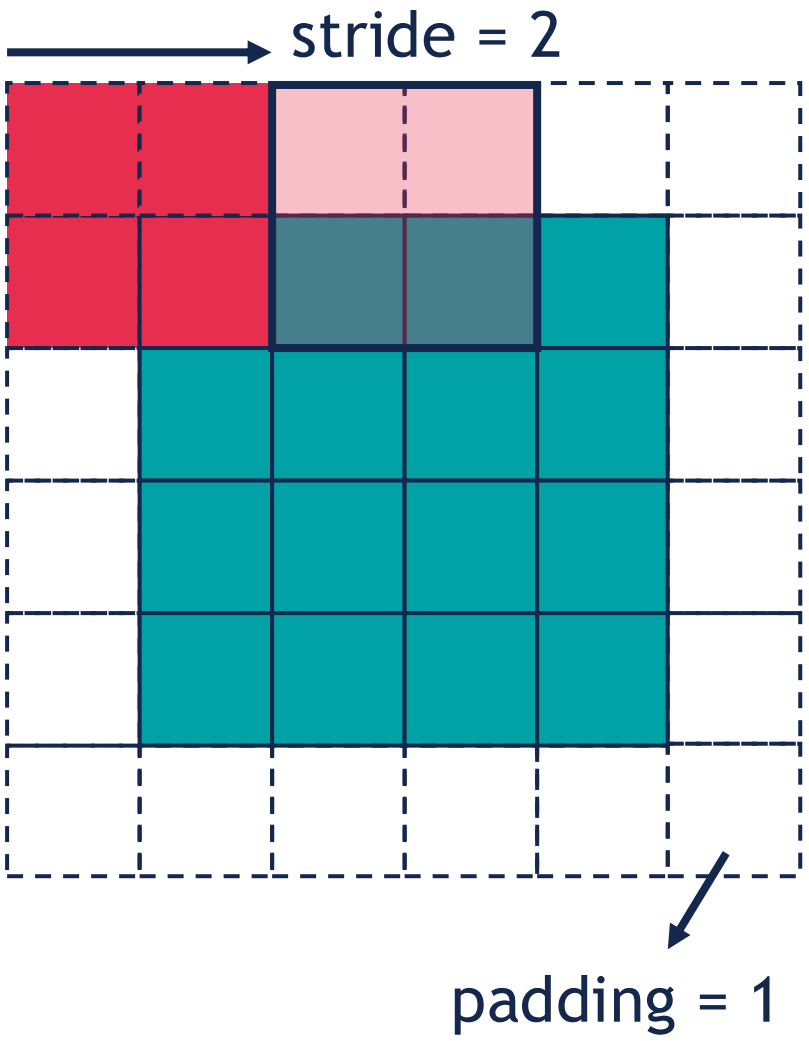
https://medium.com/nerd-for-tech/all-about-convolutions-331471b9e5e5

Padding and stride



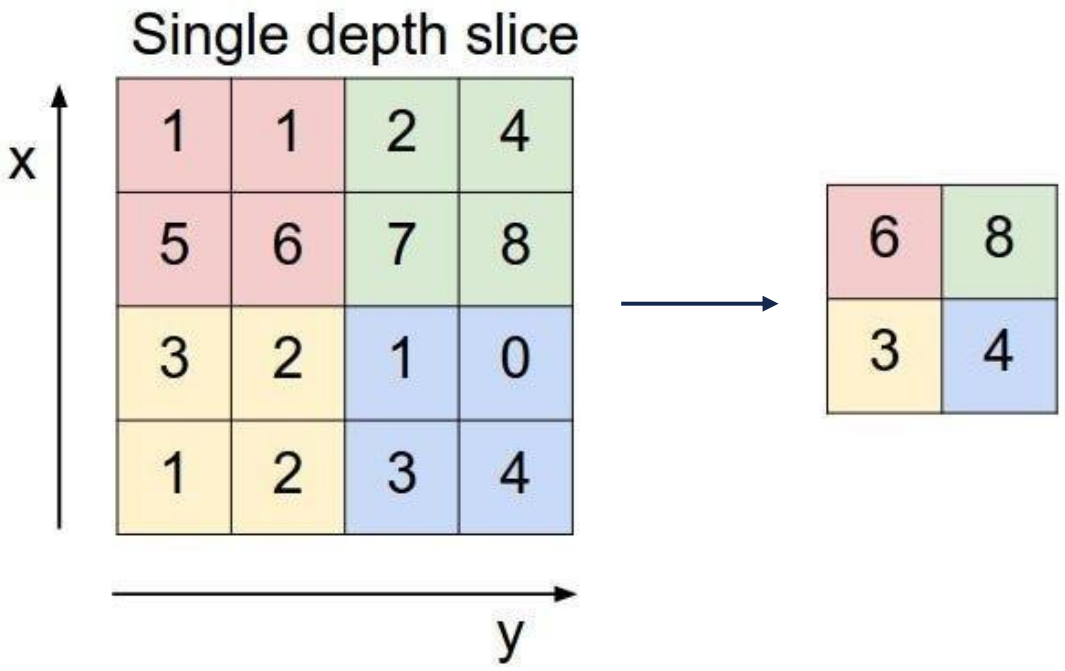
- Padding – can be used to maintain the same dimensions in the output
- Stride - how many pixels the filter moves in each step

Padding and stride



- Padding – can be used to maintain the same dimensions in the output
- Stride - how many pixels the filter moves in each step

Pooling

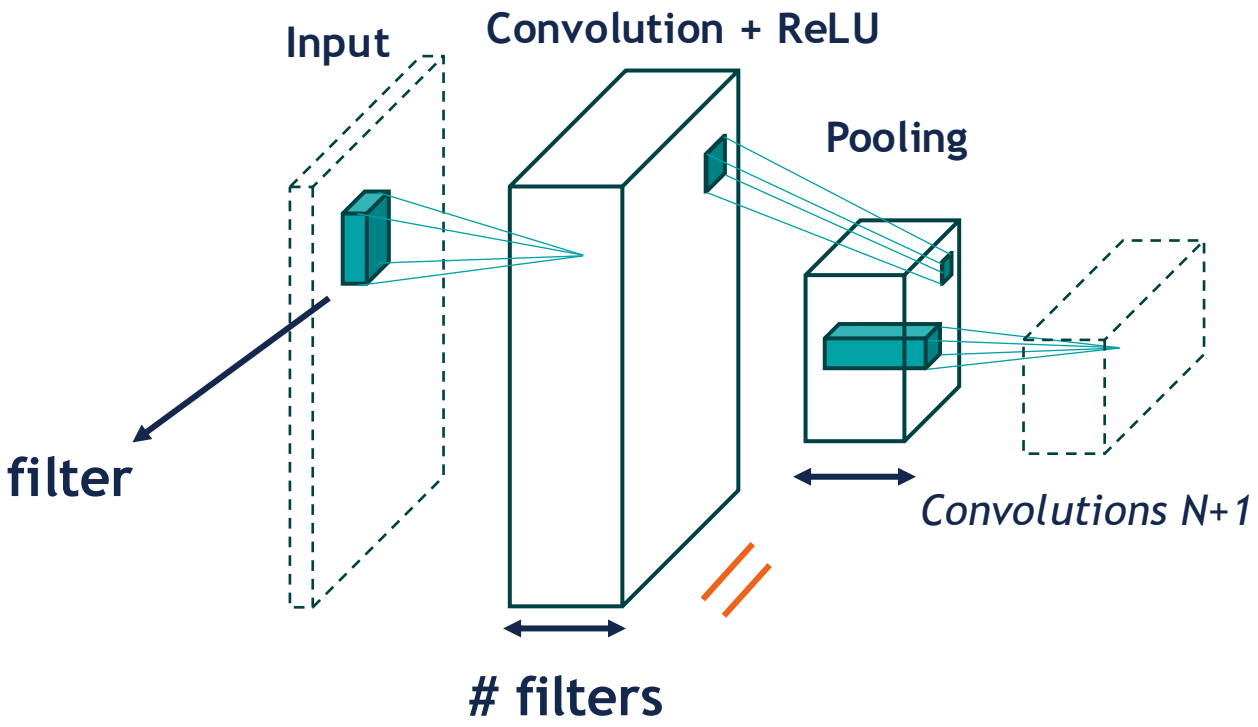


- Downsampling the filter output
- Reduced computation vs information loss
- Max pooling or average pooling

https://medium.com/nerd-for-tech/all-about-convolutions-331471b9e5e5

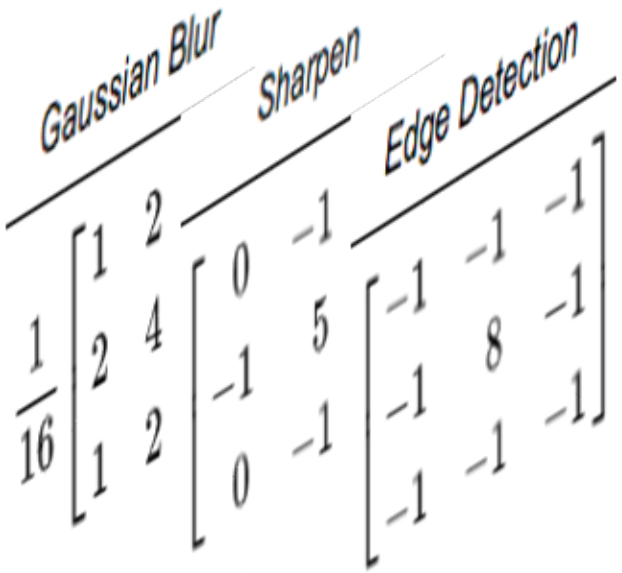
Convolutional block

1. **Convolution** (filters)
   - Padding/stride
   - Number of filters
2. **Activation** (ReLU)
3. **Pooling**
   - Max/average

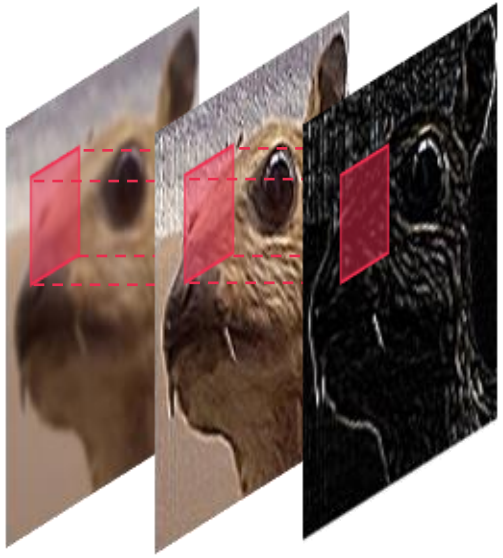


Kernel convolution

Input image



Layer 1 (3 filters)



Layer 2 (depth = 3)

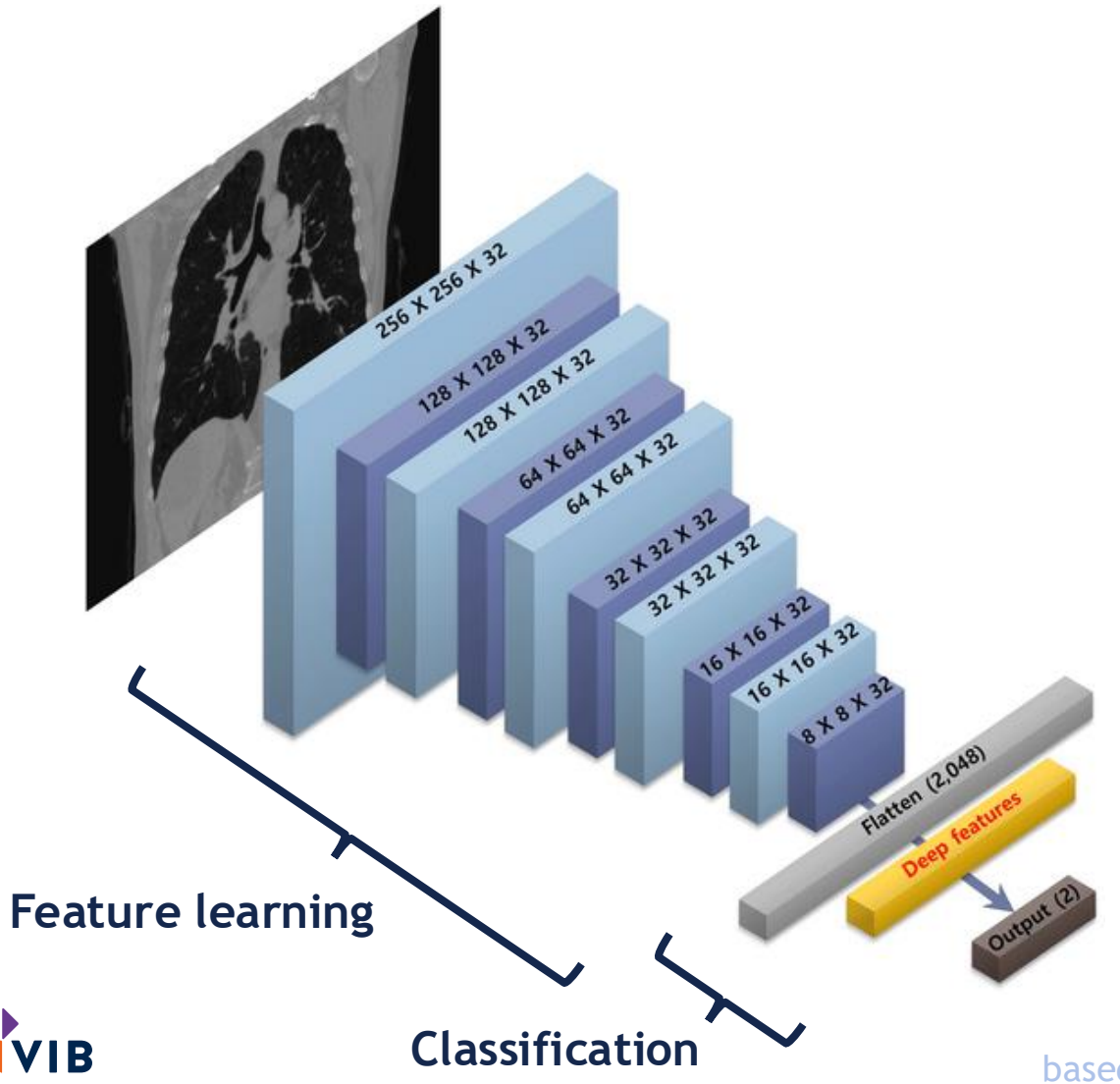
Introduction to CNNs

Kernel convolution

Channels and dimensions

CNN architectures

CNN architecture

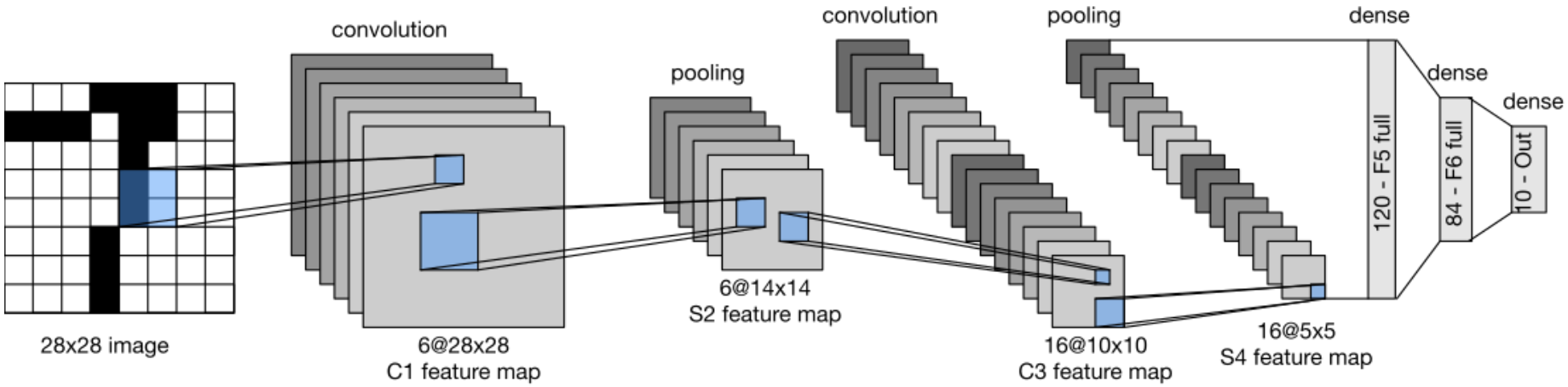


- CNN = stack of convolutional blocks (filter + ReLU + pooling)
- Filters are learned automatically
- Multiple filters can be applied to the same image
- Hierarchical learning (deeper filters learn more complex features)
  e.g. edges → shapes → objects

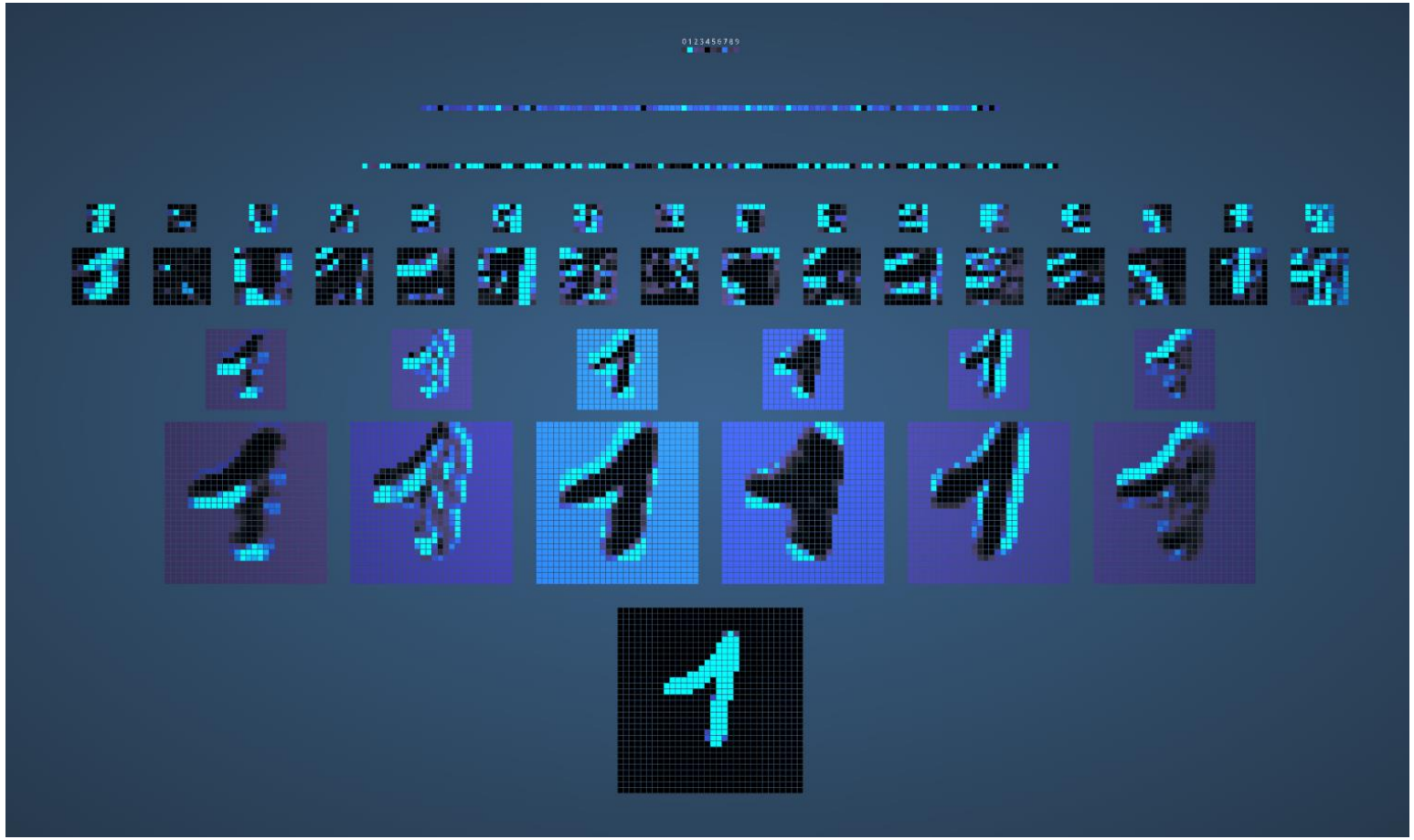
https://www.researchgate.net/publication/353459300_Deep_radiomics-based_survival_prediction_in_patients_with_chronic_obstructive_pulmonary_disease

LeNet (1998)

- First successful application of CNNs
- But: computationally too expensive (no GPUs)



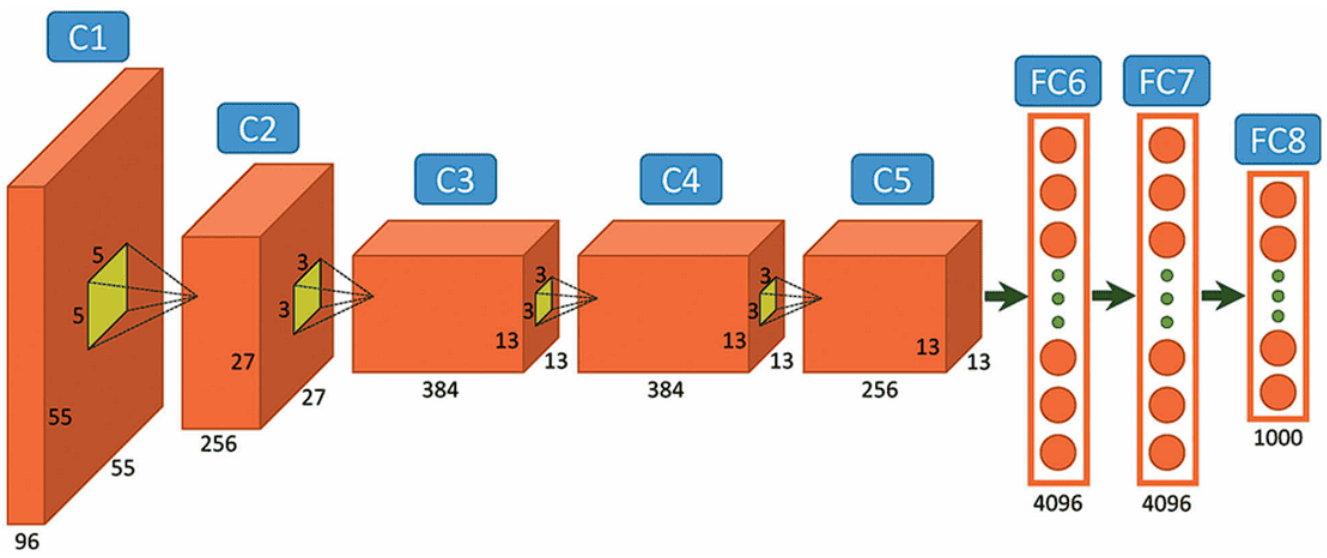
LeNet (1998)



https://adamharley.com/nn_vis/cnn/2d.html

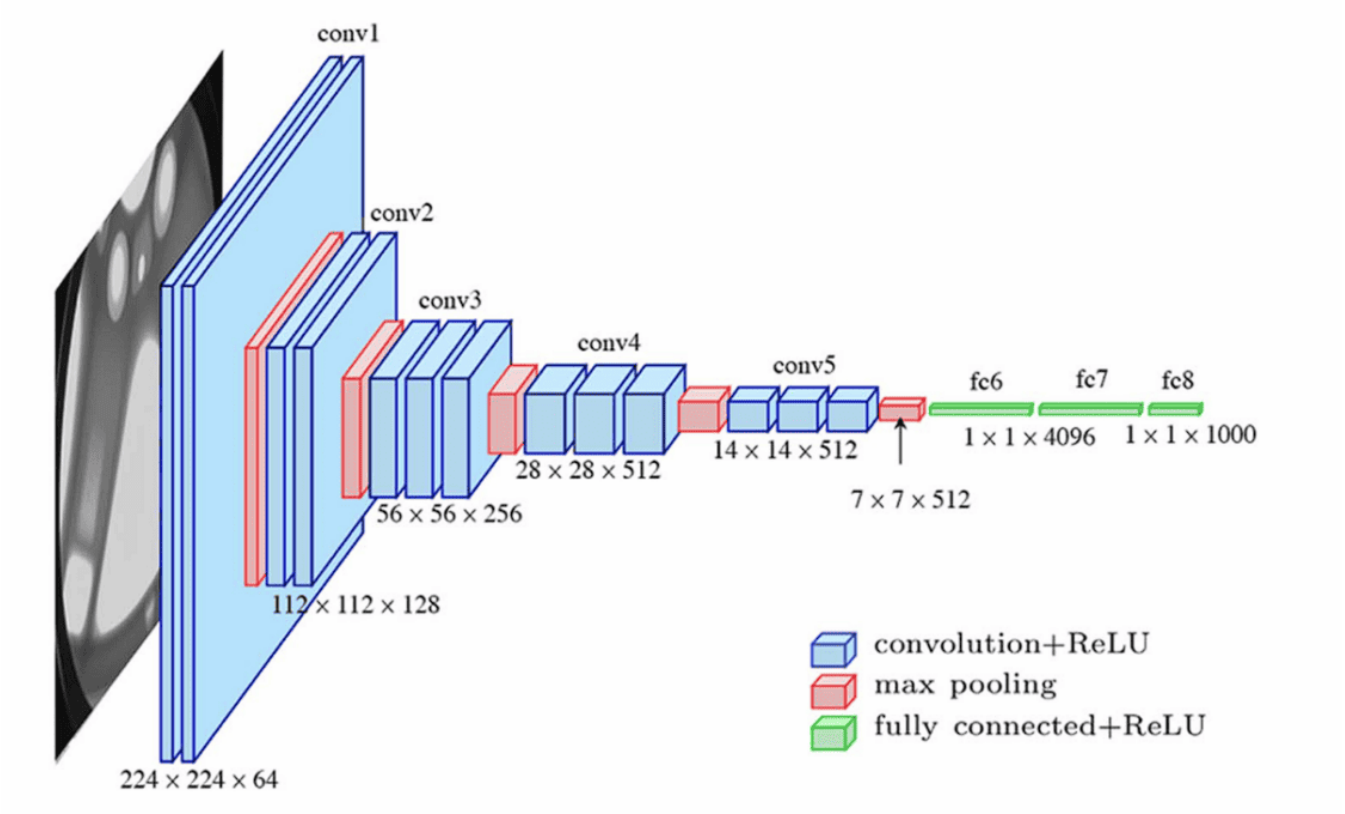
AlexNET (2012)

- GPUs and optimized training → ”deep” CNNs at scale
- Won ImageNet by a huge margin



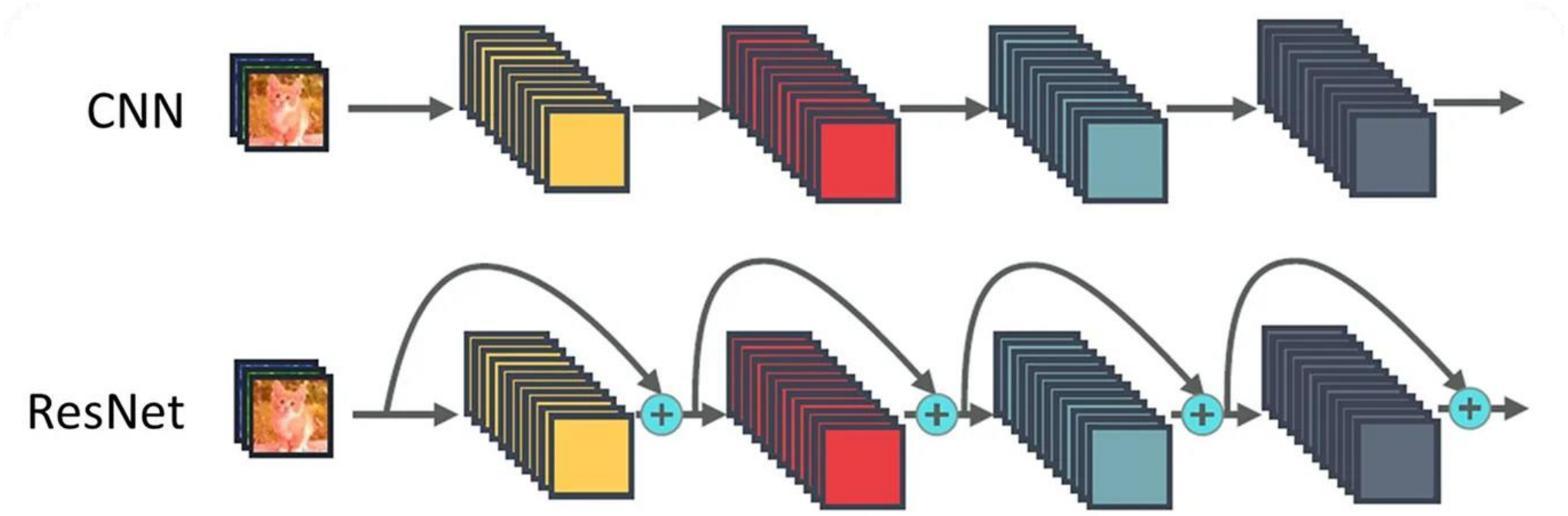
VGG (2014)

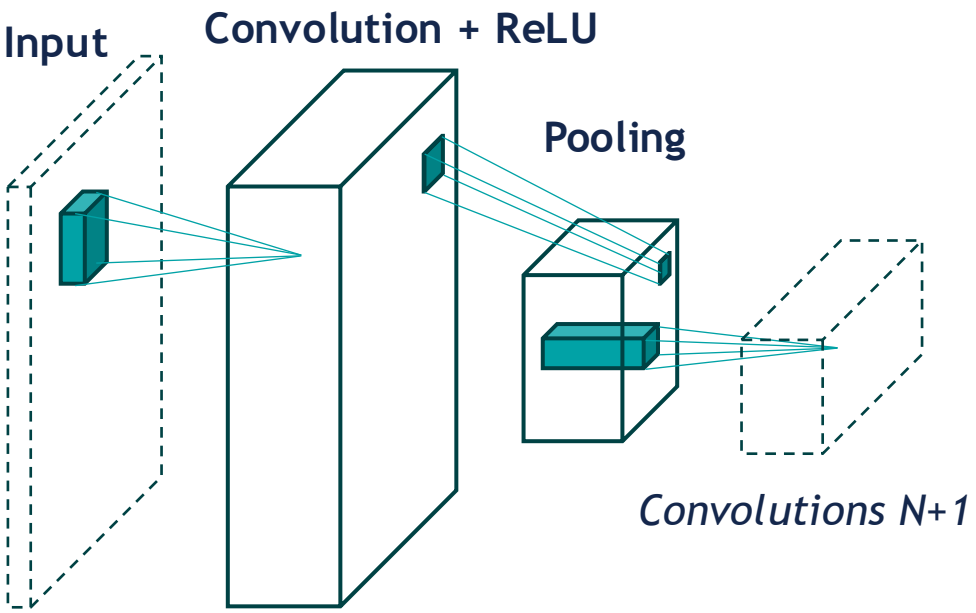
- Depth through simplicity: stacking many layers of small kernels
- Accuracy saturated at ~20 layers



ResNet (2015)

- Residual connections – network learns residuals (not full transformations)
- Enabled ultra-deep networks



CNN summary

1. CNN block
   - Filters + ReLU + pooling
2. Local information
   - Filter “sees” small patches
   - Weight sharing (fewer parameters)
3. Non-linearity + depth = complex representation
   edges → shapes → objects